

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores



Taller de diseño digital

Laboratorio 4

Código ensamblador: ARMv4

Estudiante

Rodríguez Rojas Andrés

Carné

2019279722

Profesor:

Luis Alonso Barboza Artavia

21 de mayo de 2025

Laboratorio 4: Código ensamblador: ARMv4

Fecha de asignación: 09 de mayo 2025

Fecha de entrega: 21 de mayo 2025
Profesores: Luis Barboza Artavia
Jeferson González Gómez

1. Introducción

El código ensamblador es un lenguaje de programación de bajo nivel que representa instrucciones específicas para un procesador. A diferencia de los lenguajes de programación de alto nivel, el código ensamblador está estrechamente vinculado a la arquitectura del procesador y utiliza mnemónicos y códigos de operación para representar las operaciones que la CPU debe realizar. Cada instrucción en código ensamblador se traduce directamente a una instrucción de máquina ejecutable por el procesador.

ARMv4 es una arquitectura de procesador desarrollada por ARM Holdings a principios de la década de 1990, conocida por su eficiencia energética y versatilidad. Introdujo características importantes como Thumb, una tecnología de instrucción de 16 bits para mejorar la densidad de código y el rendimiento en dispositivos con recursos limitados. ARMv4 fue ampliamente adoptada en una variedad de dispositivos móviles y embebidos, sentando las bases para el éxito continuo de ARM en la industria de semiconductores.

En este laboratorio el estudiante resolverá problemas mediante el uso de código ensamblador de ARMv4.

2. Investigación

Esta evaluación es **individual**. Para el desarrollo de este laboratorio se deben responder las siguientes preguntas.

1. Investigue los tipos de instrucción que tiene ARMv4. Nombre dos ejemplos de cada tipo y un posible uso.
2. Explique el set de registros que tiene ARM. ¿Cuál es el contenido de cada uno?
3. Explique el funcionamiento del *branch*.
4. ¿Cómo se implementa un condicional (*if/else*) en ARMv4?

5. Explique el procedimiento para transformar el código en ensamblador a binario. Investigue herramientas que realizan el proceso.
6. Explique la diferencia de *little endian* y *big endian*. ¿Cuál es su implicación cuando pasan las instrucciones a lenguaje máquina (binario)?

3. Ejercicios Prácticos

A continuación se presentan 3 ejercicios prácticos, los cuales debe resolver de manera completa **individualmente**. Para cada ejercicio debe presentar el código ensamblador en ARMv4, prueba del código ensamblador en herramienta correspondiente (e.g., [VisUAL](#), [CPUlator](#), etc.) y el archivo en lenguaje máquina (binario) equivalente.

3.1. Problema 1

Asuma que se tiene un arreglo con 10 valores y una constante definida por cada estudiante. Realice el código ensamblador (ARMv4) para el siguiente pseudocódigo:

```
int [] array = {...}
int y =

for (i = 0, 1, ..., 10)
    if array[i] >= y
        array[i] = array[i] * y
    else
        array[i] = array[i] + y
```

3.2. Problema 2

Realice el código ensamblador (ARMv4) para calcular factorial de un número X . Es decisión propia cómo se manejará el número X en el código. Realice la demostración para 3 valores de X distintos.

3.3. Problema 3

Suponga que un procesador tiene anexado un contador, que representa la posición en pantalla (VGA) de un *sprite*, así como un teclado que debe controlar dicha posición cuando se recibe la tecla de flecha superior (para aumentar el contador) y la flecha inferior (para disminuirlo). Asuma que el valor de la tecla se encuentra siempre disponible en la dirección de memoria 0x1000, y el

contador se encuentra mapeado en la dirección 0x2000. Los valores en hexadecimal de las teclas son:

Flecha de arriba -> 0xE048

Flecha de abajo -> 0xE050

Con base en lo anterior, realice un programa usando ARMv4 que constantemente lea el valor de la tecla y actualice el contador correspondiente, aumentándolo o disminuyéndolo dependiendo de cuál flecha se presionó. El programa NO debe cambiar el contador en caso de que la tecla presionada no sea una de las dos opciones válidas.

Nota: Para efectos de la prueba del código, simule el valor de la tecla, escribiendo en la dirección de memoria correspondiente de manera manual para al menos 5 iteraciones del bucle. Se debe probar las dos teclas válidas, así como al menos una tecla inválida, para asegurarse de que el sistema funciona correctamente.

4. Evaluación

La evaluación de este laboratorio será individual, por medio de un informe escrito y repositorio donde se encuentren los códigos fuente (ensamblador y lenguaje máquina) con la solución a los problemas planteados. En el informe escrito deberán presentarse las respuestas de la sección de investigación, así como capturas de pantalla de las pruebas (simulación en herramienta elegida) para cada uno de los ejercicios prácticos.

La entrega se debe realizar por medio del TEC-Digital en la pestaña de evaluación. No se aceptan entregas extemporáneas después de la fecha de entrega a las 11:59 pm como máximo. **Los documentos serán sometidos a control de plagios.**

I. INTRODUCCIÓN

El lenguaje ensamblador es fundamental en el desarrollo de sistemas embebidos y aplicaciones de bajo nivel, ya que permite un control directo sobre el hardware del procesador. En este laboratorio, se explora la arquitectura ARMv4, conocida por su eficiencia energética y amplio uso en dispositivos móviles y embebidos. A través de la implementación de código ensamblador, se abordarán problemas prácticos que demuestran el manejo de instrucciones, registros, estructuras de control como condicionales y bucles, el acondicionamiento a la sintaxis pertinente para un lenguaje de bajo nivel.

II. INVESTIGACIÓN

1. Investigue los tipos de instrucción que tiene ARMv4. Nombre dos ejemplos de cada tipo y un posible uso.

En ARMv4 existen varios tipos de instrucciones:

- Instrucciones de transferencia de datos: Son aquellos que mueven datos entre registros y memoria. Utilizan palabras reservadas como *LDR* y *STR* [3].
- Instrucciones aritméticas y lógicas: Realizan operaciones y procesos matemáticos entre registros y bits. Poseen palabras reservadas como *ADD* (para una suma) y *AND* (para efectuar comparaciones AND entre los bits de dos registros) [1].
- Instrucciones de control de flujo: Capaces de modificar y alterar el flujo de ejecución del programa, se identifican por palabras reservadas como *B* (para saltos sin condición) y *BEQ* (para saltos condicionales) [3].
- Instrucciones de multiplicación: Para multiplicaciones enteras, se utilizan mediante las palabra reservada *MUL* [1].
- Instrucciones de comparación: Se implementan mediante la sintaxis *CMP*, comparan dos registros y permiten la existencia de bucles y estructuras condicionales [2].

2. Explique el set de registros que tiene ARM. ¿Cuál es el contenido de cada uno?

Existen los registros generales u aptos para guardar registros temporales. Se utilizan para operaciones aritméticas o para reservar direcciones de memoria, corresponden a los registros entre *R0* y *R12* [1].

También existe un registro dedicado al manejo de llamadas a funciones, se le llama “stack pointer” y contiene la dirección del inicio del stack (registro *R13*) [1].

Además, posee un registro *R14* dedicado a almacenar una dirección de retorno, lo cual resulta de suma utilidad a la hora de implementar bucles y agilizar el trabajo de los mismos. Es modificable según la palabra reservada *BL* [1].

Finalmente, ARMv4 incluye un registro *R15* o “program counter” que almacena la dirección de la instrucción que se ejecuta en un momento dado, adicionando 8 a dicha dirección de memoria [1].

3. Explique el funcionamiento del branch.

El branch permite realizar alteraciones en el flujo de un programa, según se explicaba en la instrucción propia de este elemento (aquella con la palabra reservada *B*). Permite realizar “saltos” entre diferentes secciones del programa, cuyas direcciones de memoria se guardan en etiquetas determinadas por el desarrollador.

Si se requiere regresar a un punto particular del flujo sin importar el estado del programa o el valor de ninguna de las variables del programa, es posible hacerlo según la instrucción *B*, sin embargo, si se requiere hacer un salto condicionado, ya sea por el valor de una variable o por una condición específica del sistema, es posible hacerlo utilizando las palabras reservadas *BEQ*, para saltos en los que se requiera una igualdad entre registros o un resultado positivo para una condición particular, o bien, *BNE* en el caso de requerirse una inequidad.

4. ¿Cómo se implementa un condicional (if /else) en ARMv4?

Para la implementación de un condicional en ARMv4, es necesario aplicar algunos de los conceptos definidos anteriormente, como lo son la alteración del flujo de ejecución del programa (o uso del branch), o el uso de instrucciones de comparación.

Inicialmente, se debe identificar y establecer los diferentes procesos que debe realizar la estructura condicional y las condiciones que se deben cumplir para que cada uno de dichos procesos se lleve a cabo. A su vez, es necesario definir un identificador de branch, con el objetivo de poder acceder a cada uno de los bloques de código de manera ágil y desde cualquier sección del código.

Una vez hecho esto, y haciendo uso del comparador *CMP*, se deben evaluar las condiciones que posee la estructura condicional a implementar y, utilizando la sintaxis *BEQ* es posible volver a la etiqueta que lleva a cabo el proceso necesario en caso de que el condicional tenga un resultado positivo, o bien *BNE* para volver al bloque de código que ejecuta el proceso que se debe desarrollar en caso de que la condición no se cumpla (else).

5. Explique el procedimiento para transformar el código en ensamblador a binario. Investigue herramientas que realizan el proceso.

Para la traducción de código ensamblador a lenguaje máquina, es necesario el uso de un software ensamblador o “traductor”, tales como GNU Assembler, VisUAL o CPULator, los cuales se encargan de convertir el lenguaje ensamblador programable a su respectivo equivalente en formato binario.

Para ello, inicialmente es necesario contar con un programa en ensamblador con instrucciones de funcionamiento claras y definidas, posteriormente dichas herramientas de software proceden a reemplazar cada “mnemónico” por su equivalente “opcode” binario, así como las direcciones de memoria necesarias para los elementos necesarios dentro del bloque de instrucciones. Una vez completado dicho proceso, se genera un archivo de salida cuyo código es binario y se encuentra disponible para su consecuente depuración [1].

6. Explique la diferencia de little endian y big endian. ¿Cuál es su implicación cuando pasan las instrucciones a lenguaje máquina (binario)?

Little endian y big endian corresponden a dos formatos de almacenamiento de bytes con un comportamiento que se podría considerar opuesto o inverso.

En el caso de little endian, este formato almacena el byte menos significativo se almacena en la dirección más baja de memoria, mientras que, para el formato big endian, el byte más significativo se almacena en la dirección más baja, por lo que, tan solo con realizar un análisis cualitativo de su funcionamiento es posible deducir que los datos se almacenan inversamente siguiendo cada uno de los métodos anteriores.

En consecuencia, si un software ensamblador lee e interpreta los datos almacenados utilizando un formato incorrecto, son esperables fallos en el direccionamiento de las variables y estructuras, y consecuentemente, un funcionamiento anómalo o diferente al esperado según el software programado.

III. ANÁLISIS DE RESULTADOS

III-A. Problema 1

A continuación se adjuntan las evidencias de funcionamiento para un programa que genera la multiplicación o suma de la posición de un arreglo con un valor y . Para efectos de este problema, se trabaja con un arreglo $array = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ y un valor $y = 2$.

Inicialmente se aprecia como todos los registros se encuentran inicializados en 0, como se observa en la figura 1.

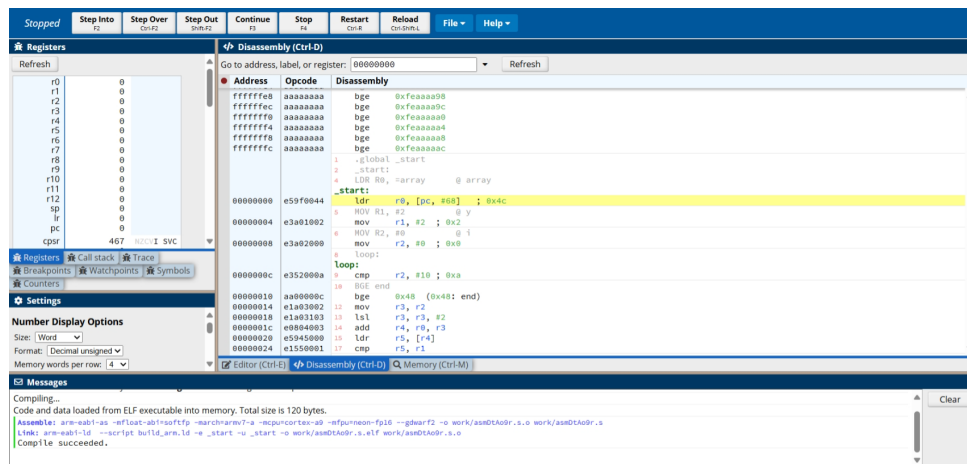


Figura 1: Estado inicial del programa del problema 1.

Para la iteración $i = 0$, se observa cómo el registro $R6$ toma el valor de 3, debido a que el elemento en la posición 0 del arreglo es equivalente a 1, por lo que, al compararse con $y = 2$ y ser menor, se procede a la condición del `else`, asignando temporalmente al registro $R6$ el valor de $R6 = 1 + 2 = 3$, tal como se observa en la figura 2.

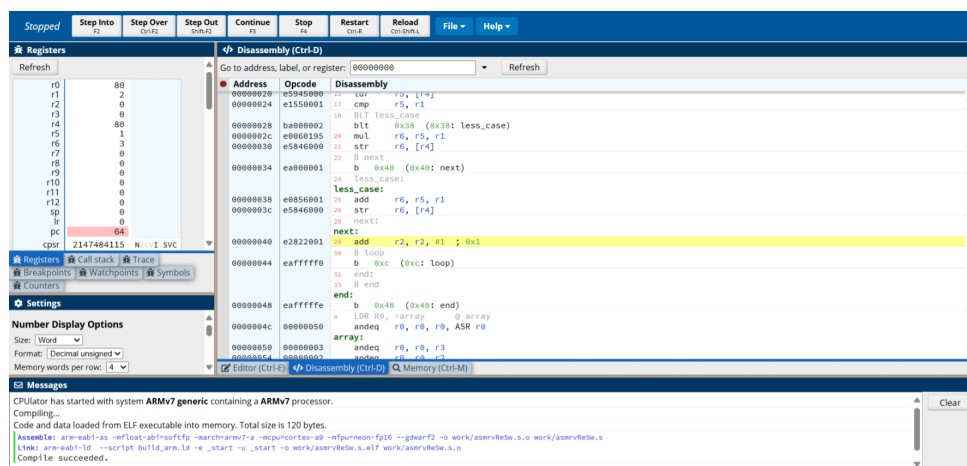


Figura 2: Primera iteración del problema 1 ($i = 0$ y $array[i] = 1$).

Para la segunda iteración del programa ($i = 1$), el valor en dicha posición del arreglo corresponde a 2, por tanto, se cumple la condición del `if` y se procede a multiplicar $2 \cdot 2$, por lo que $R6 = 4$, como se observa en la figura 3.

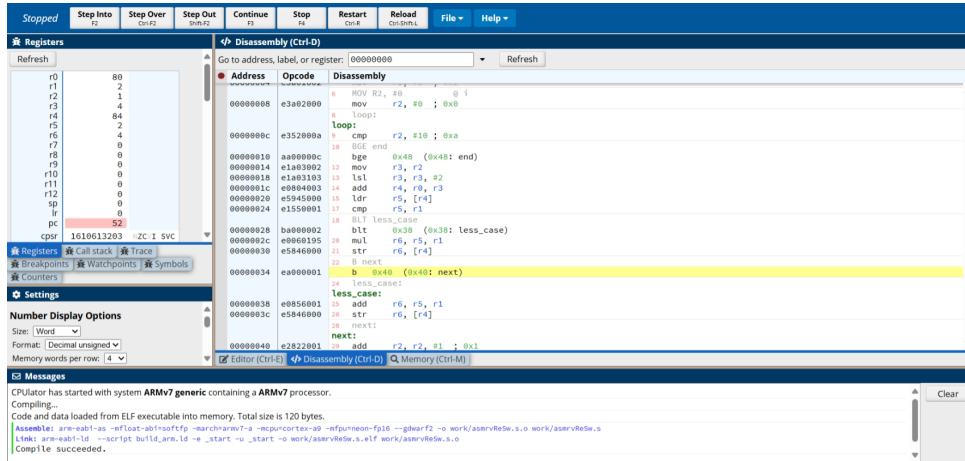


Figura 3: Segunda iteración del problema 1 ($i = 1$ y $array[i] = 2$).

Al finalizar el programa, se observa en la figura 4 el resultado de la última iteración, donde $i = 10$, lo que causa la salida del condicional y $array[i] = 10$, por lo que, al igual que el caso anterior, al cumplirse la condición del condicional, se duplica el valor del elemento en la posición del arreglo, de modo que $R6 = 2 \cdot 10 = 20$.

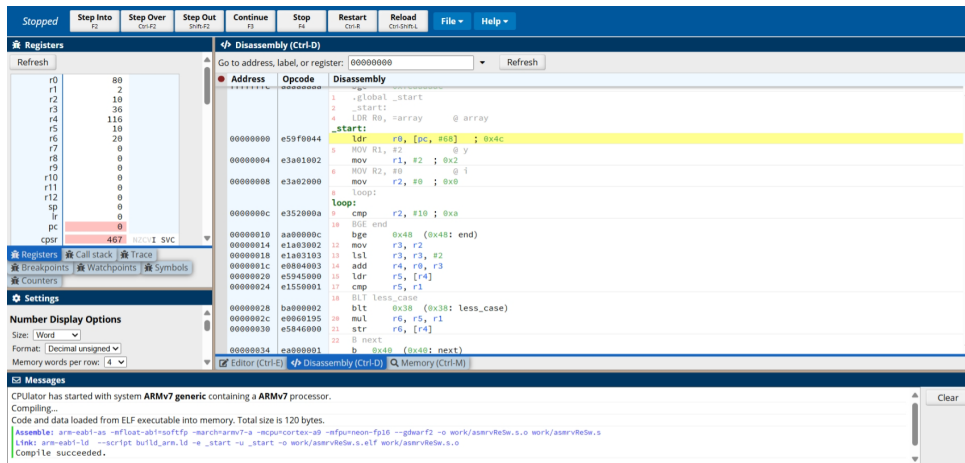


Figura 4: Última iteración del problema 1 ($i = 9$ y $array[i] = 10$).

III-B. Problema 2

A continuación se presentan las evidencias de funcionamiento para un programa que calcula el factorial de un número entero positivo x , utilizando un ciclo `loop` que decrece el valor de x hasta alcanzar 1, acumulando el resultado parcial en un registro.

El programa inicia con los registros principales inicializados en cero, y el valor de x se carga en el registro R1, mientras que el resultado acumulado se inicializa con 1 en el registro R0.

En la figura 5 se muestra la ejecución del programa para el caso $x = 1$. Dado que el valor inicial de x ya es igual a 1, la condición del bucle no se cumple, por lo que no se realiza ninguna multiplicación y el resultado permanece como 1.

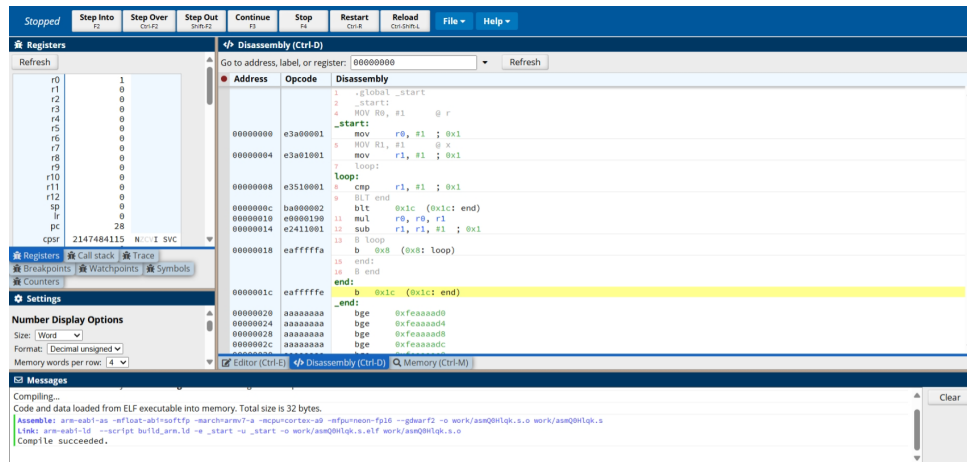


Figura 5: Ejecución del programa con $x = 1$.

En la figura 6 se muestra la ejecución para el valor $x = 8$. Durante la ejecución, el registro R1 se va decrementando desde 8 hasta 1, y en cada iteración el valor de R0 se actualiza con el producto de su valor anterior por el valor actual de R1, resultando finalmente en $R0 = 40320$, que corresponde a $8!$.

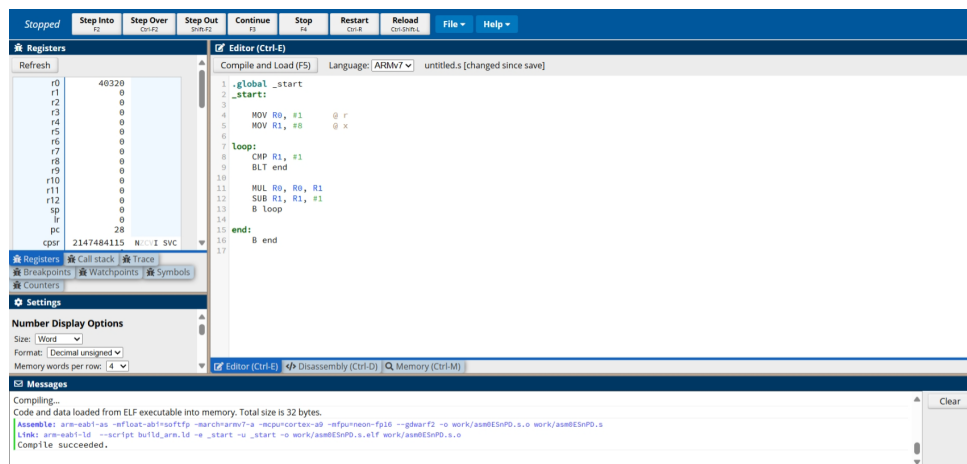


Figura 6: Ejecución del programa con $x = 8$.

Finalmente, en la figura 7, se ejecuta el programa con $x = 10$, por lo que el bucle se repite diez veces. El valor en R0 se actualiza sucesivamente con los productos $1 \cdot 10$, $10 \cdot 9$, $90 \cdot 8$, y así hasta llegar a 1, dando como resultado final $R0 = 3628800$, valor equivalente a $10!$.

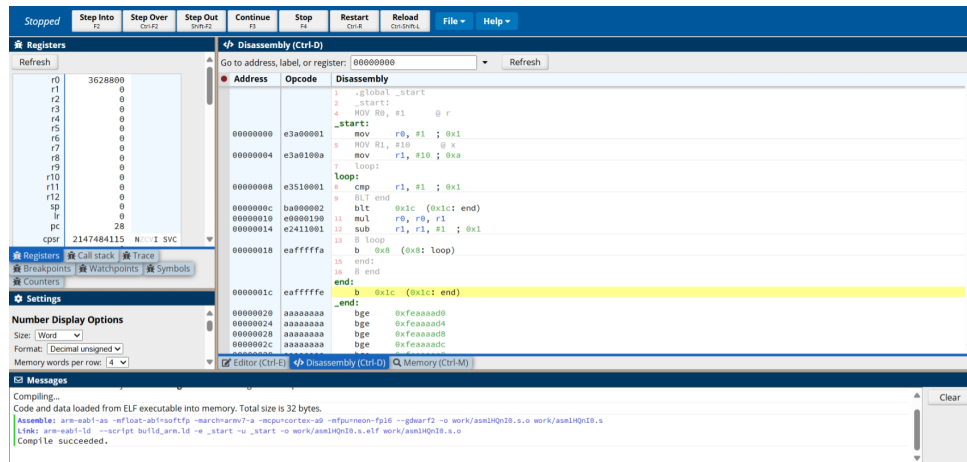


Figura 7: Ejecución del programa con $x = 10$.

III-C. Problema 3

Este programa simula el comportamiento de un sprite cuya posición debe ser controlada a través de un teclado, específicamente por medio de las teclas de flecha superior ($\uparrow$) y flecha inferior ($\downarrow$), cuyas representaciones en código hexadecimal son $0xE048$ y $0xE050$, respectivamente. El contador de posición del sprite se almacena en un registro y se modifica únicamente cuando se detecta una de estas dos entradas válidas.

Dado que el entorno de ejecución (CPUlator) no permite la captura directa de entradas desde teclado, se simula el funcionamiento mediante un arreglo denominado `inputs`, el cual contiene cinco valores de entrada representando teclas presionadas: tres flechas arriba, una flecha abajo y un valor inválido.

En la figura 8 se muestra el estado inicial de los registros. El contador de posición del sprite (R5) se encuentra inicializado en cero. El arreglo de entradas es procesado una por una, comparando cada valor con las teclas válidas y actualizando el contador en consecuencia.

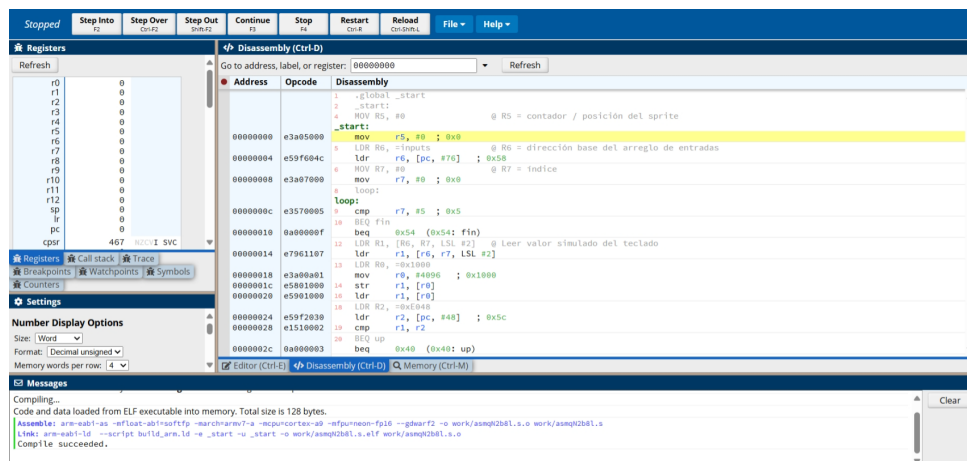


Figura 8: Estado inicial del problema 3.

Durante la primera iteración, el valor leído es $0xE048$, correspondiente a la flecha superior, por lo que el contador incrementa en uno ($R5 = 1$), como se muestra en la figura 9.

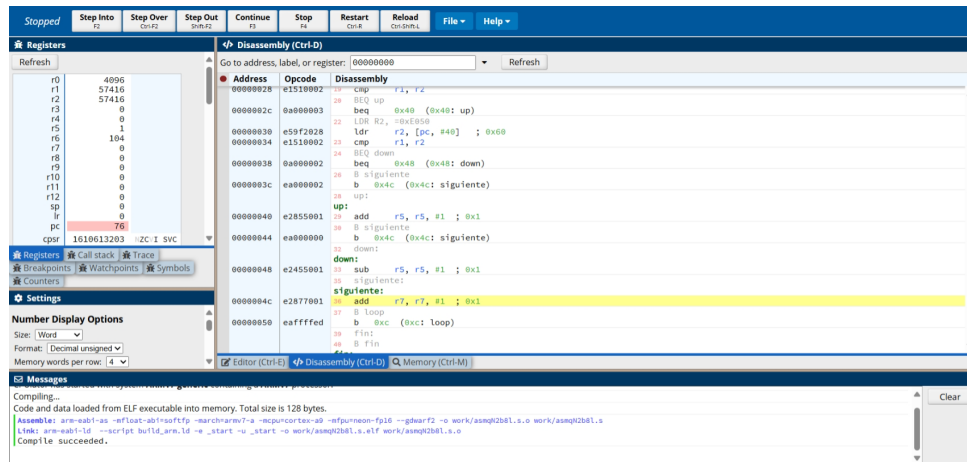


Figura 9: Primera iteración del problema 3.

En la segunda iteración se lee el valor $0xE050$, correspondiente a la flecha inferior, por lo que se decrementa el contador ($R5 = 0$), como se observa en la figura 10. En la tercera y quinta iteración se detecta nuevamente la flecha superior, incrementando el contador cada vez. En la cuarta iteración se presenta un valor inválido ($0x1234$), por lo que el contador no se modifica.

Finalmente, tras procesar las cinco entradas del arreglo, el valor del contador es 2, como se aprecia en la figura 10.

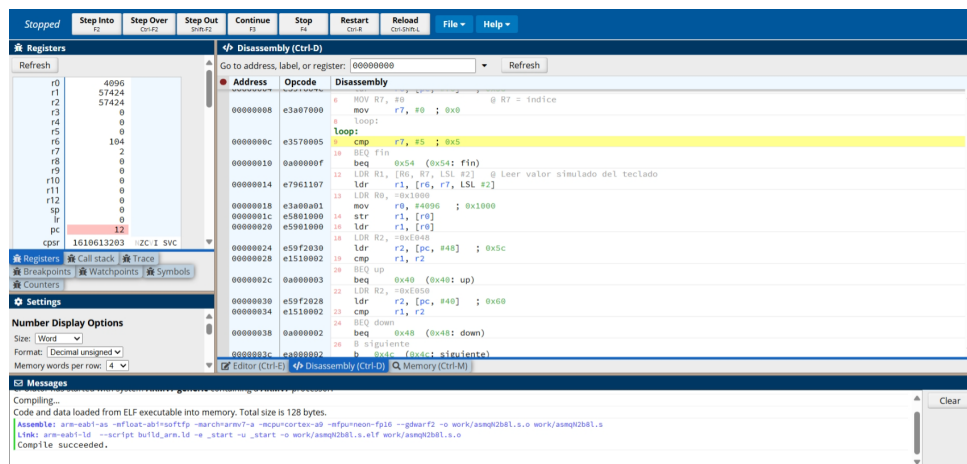


Figura 10: Segunda iteración del problema 3, con valor del contador igual a 0.

IV. CONCLUSIONES

El desarrollo de programas en lenguaje ensamblador para la arquitectura ARMv4 permitió comprender a profundidad el funcionamiento de instrucciones de bajo nivel, tales como transferencias de datos, operaciones aritméticas, condicionales y control de flujo. La implementación práctica de problemas mediante CPUlator facilitó la simulación de instrucciones y el análisis paso a paso del comportamiento de los registros, así como la manipulación de memoria mapeada.

Además, la investigación teórica sobre el conjunto de instrucciones, registros y aspectos relevantes como el formato little endian y big endian reforzó el conocimiento sobre cómo el hardware interpreta y ejecuta las instrucciones. Esto evidencia la importancia de una programación precisa y el dominio de los conceptos fundamentales de la arquitectura para asegurar el funcionamiento correcto y predecible del sistema.

V. REFERENCIAS BIBLIOGRÁFICAS

- [1] David Money Harris y Sarah L. Harris. *Digital Design and Computer Architecture*. 2nd. Morgan Kaufmann, 2012. ISBN: 978-0-12-394424-5.
- [2] ARM Limited. *ARM7TDMI Technical Reference Manual*. ARM Ltd., 1996.
- [3] David A. Patterson y John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. 5th. Morgan Kaufmann, 2013. ISBN: 978-0-12-407726-3.